Ramakrishna Mission Vivekananda Centenary College

Rahara, Kolkata-118

HPC Lab Assignment

Registration No: A01-1112-217-022-2025

Semester: II

Roll: 23

Department: Computer Science

Subject Code: PGCMSCC 2.2

Subject Name: Computer Architecture & High Performance Computing

Academic Year: 2025–2026

Contents

1 Write a program in python to find the rank of random generated matrix using MPI. 3
1.1 Python Code 3
1.2 Output 4

2 Write a program in python to find the eigen value of random generated matrix using MPI. 5
2.1 Python Code 5
2.2 Output 6

3 Write a program in python to find the eigen vectors of random generated matrix using MPI. 7
3.1 Python Code 7
3.2 Output 8

4 Write a program in python to find the inverse of random generated matrix using MPI. 9
4.1 Python Code 9
4.2 Output 11

5 Write a program in python to find the linear interpolation of randomly generated data using MPI. 12
5.1 Python Code 12
5.2 Output 13

6 Write a program in python to find the cubic interpolation of randomly generated data using MPI. 14
6.1 Python Code 14
6.2 Output 15

7 Write a program in python to find the cosine interpolation of randomly generated data using MPI. 16
7.1 Python Code 16
7.2 Output 17

8 Write a program in python to find the Nearest Neighbor interpolation of randomly generated data using MPI. 18
8.1 Python Code 18
8.2 Output 19

9 Write a program in python to implement KNN Algorithm and find the accuracy using a dataset using MPI. 20
9.1 Python Code 20
9.2 Output 21

10 Write a program in python to implement K Means Clustering Algorithm and find the accuracy using a dataset using MPI. 22
10.1 Python Code 22

10.2 Output 25

11 Write a program to implement Bubble sort using OpenMP in C Programming 26
11.1 C Code 26
11.2 Output 27

12 Write a program to implement two matrix addition using OpenMP in C Programming 28
12.1 C Code 28
12.2 Output 29

13 Write a program to implement 2d heat equation using OpenMP and MPI in C Programming 30
13.1 C Code 30
13.2 Output 32

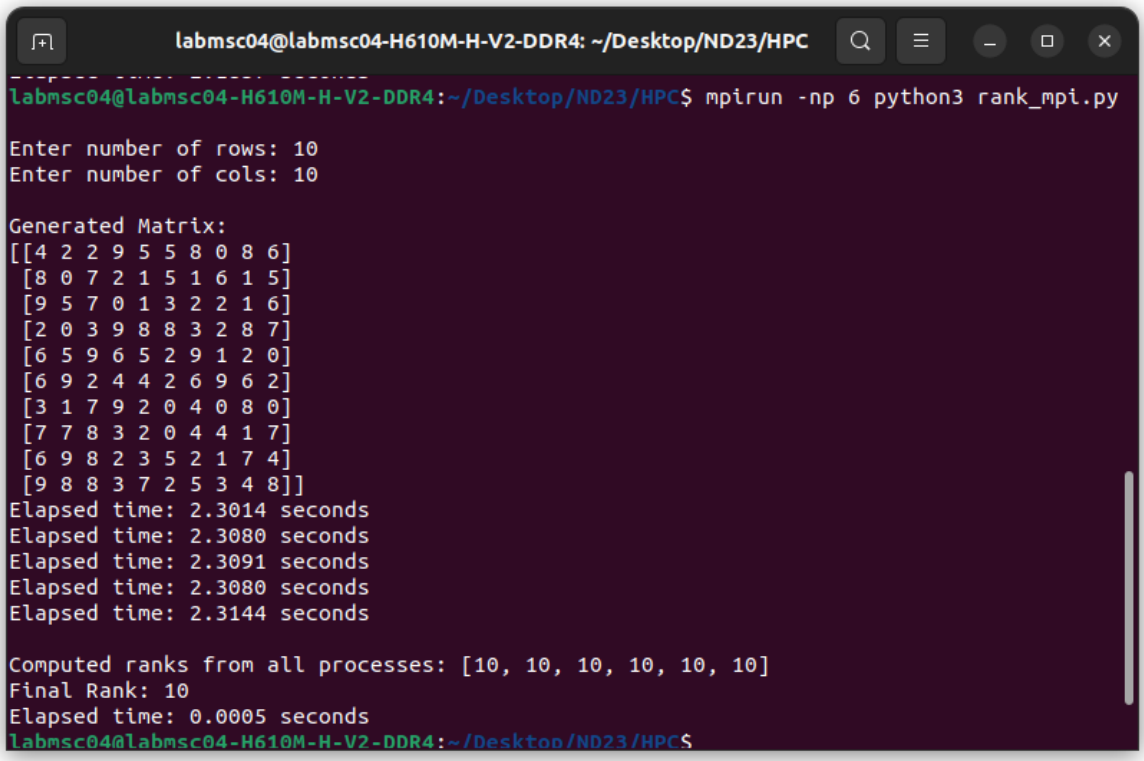
1 Write a program in python to find the rank of random generated matrix using MPI.

1.1 Python Code

```python
from mpi4py import MPI
import random
import numpy as np
import time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()
# ---------- Matrix Rank Function (Gaussian Elimination) ----------
def matrix_rank(A):
    A = A.astype(float)
    m, n = A.shape
    rank_val = 0
    row = 0

    for col in range(n):
        pivot = None
        for r in range(row, m):
            if abs(A[r, col]) > 1e-10:
                pivot = r
                break
        if pivot is not None:
            # Swap rows
            A[[row, pivot]] = A[[pivot, row]]
            # Eliminate below
            for r in range(row + 1, m):
                factor = A[r, col] / A[row, col]
                A[r, col:] -= factor * A[row, col:]
            row += 1
            rank_val += 1
        if row == m:
            break
    return rank_val
# ---------- Create Random Matrix ----------
def create_matrix(a, b):
    return np.array([[random.randint(0, 9) for _ in range(b)] for _ in range(a)
        ])
# ---------- Main Program ----------
if rank == 0:
    x = int(input("Enter number of rows: "))
    y = int(input("Enter number of cols: "))
    A = create_matrix(x, y)

    print("\nGenerated Matrix:")
    print(A)
```

```python
else:
    A = None
start_time = time.perf_counter()
# Broadcast matrix to all processes
A = comm.bcast(A, root=0)
# Each process computes rank
local_rank = matrix_rank(A)
# Gather results at root
all_ranks = comm.gather(local_rank, root=0)
end_time = time.perf_counter()
if rank == 0:
    print("\nComputed ranks from all processes:", all_ranks)
    print("Final Rank:", all_ranks[0])
elapsed_time = end_time - start_time
print(f"Elapsed time: {elapsed_time:.4f} seconds")
```

1.2 Output



2 Write a program in python to find the eigen value of random generated matrix using MPI.

2.1 Python Code

```python
from mpi4py import MPI
import random
import numpy as np
import time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# ---------- Power Iteration Eigenvalue Function ----------
def eigenvalue_power(A, iterations=100):
    n = A.shape[0]
    # Random initial vector
    v = np.random.rand(n)
    v = v / np.linalg.norm(v)
    eigenvalue = 0
    for _ in range(iterations):
        Av = np.dot(A, v)
        # Rayleigh quotient approximation
        eigenvalue = np.dot(v, Av)
        # Normalize vector
        v = Av / np.linalg.norm(Av)
    return eigenvalue

# ---------- Create Random Matrix ----------
def create_matrix(a, b):
    return np.array([[random.randint(0, 9) for _ in range(b)] for _ in range(a)
        ])

# ---------- Main Program ----------
if rank == 0:
    x = int(input("Enter number of rows: "))
    y = int(input("Enter number of cols: "))
    if x != y:
        print("Eigenvalues require a square matrix.")
        comm.Abort()
    A = create_matrix(x, y)
    if x<10:
        print(A)
else:
    A = None

start_time = time.perf_counter()

# Broadcast matrix to all processes
```

```python
A = comm.bcast(A, root=0)

# Each process computes eigenvalue (parallel demonstration)
local_eigen = eigenvalue_power(A)

# Gather results at root
all_eigen = comm.gather(local_eigen, root=0)
end_time = time.perf_counter()
if rank == 0:
    print("\nEigenvalues computed by all processes:", all_eigen)
    print("Largest Eigenvalue ", all_eigen[0])
elapsed_time = end_time - start_time
print(f"Elapsed time: {elapsed_time:.4f} seconds")
```

2.2 Output

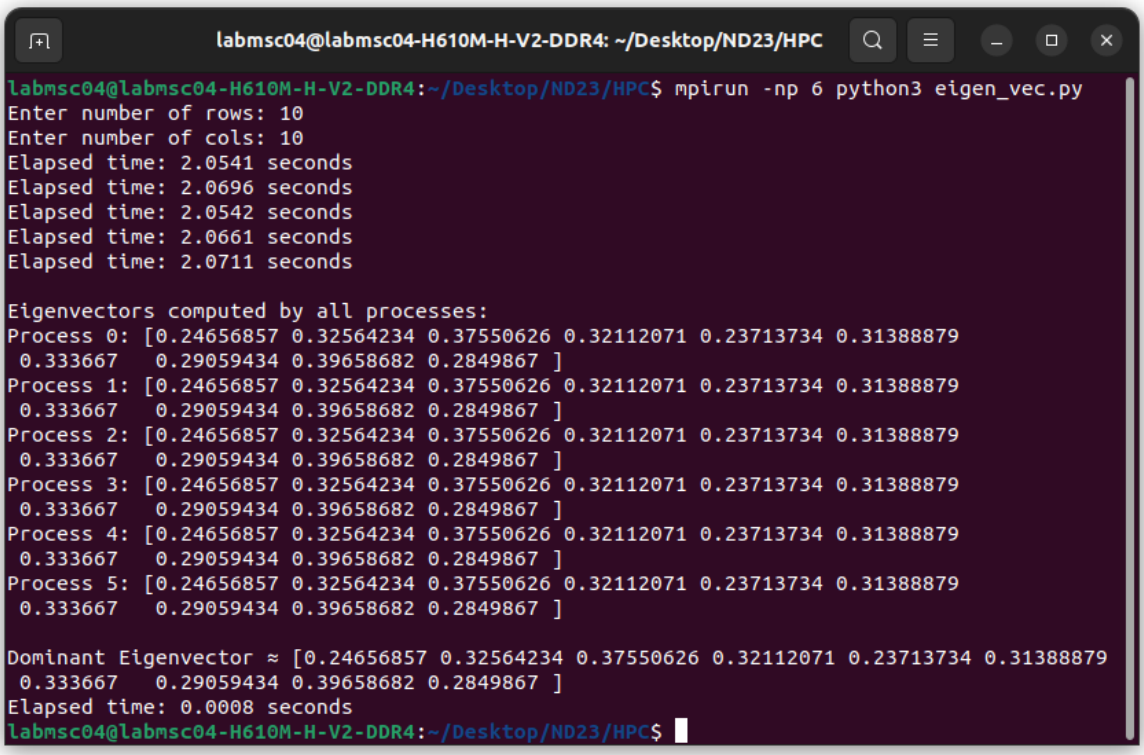
3 Write a program in python to find the eigen vectors of random generated matrix using MPI.

3.1 Python Code

```python
from mpi4py import MPI
import random
import numpy as np
import time


comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# ---------- Power Iteration Eigenvector Function ----------
def eigenvector_power(A, iterations=100):
    n = A.shape[0]
    # random initial vector
    v = np.random.rand(n)
    v = v / np.linalg.norm(v)

    for _ in range(iterations):
        Av = np.dot(A, v)

        # normalize vector
        v = Av / np.linalg.norm(Av)

    return v

# ---------- Create Random Matrix ----------
def create_matrix(a, b):
    return np.array([[random.randint(0, 9) for _ in range(b)] for _ in range(a)
        ])

# ---------- Main Program ----------
if rank == 0:
    x = int(input("Enter number of rows: "))
    y = int(input("Enter number of cols: "))

    if x != y:
        print("Eigenvectors require a square matrix.")
        comm.Abort()

    A = create_matrix(x, y)

    if x < 10:
        print("\nGenerated Matrix:")
        print(A)

```

```python
else:
    A = None


start_time = time.perf_counter()

# Broadcast matrix
A = comm.bcast(A, root=0)

# Each process computes eigenvector
local_vec = eigenvector_power(A)

# Gather results
all_vecs = comm.gather(local_vec, root=0)
end_time = time.perf_counter()
if rank == 0:
    print("\nEigenvectors computed by all processes:")
    for i, v in enumerate(all_vecs):
        print(f"Process {i}:", v)
    print("\nDominant Eigenvector ", all_vecs[0])
elapsed_time = end_time - start_time
print(f"Elapsed time: {elapsed_time:.4f} seconds")
```

3.2 Output



4 Write a program in python to find the inverse of random generated matrix using MPI.

4.1 Python Code

```python
from mpi4py import MPI
import random
import numpy as np
import time


comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()


# ---------- Matrix Inverse Function (Gauss-Jordan) ----------
def matrix_inverse(A):
    n = A.shape[0]

    # Create augmented matrix [A | I]
    I = np.identity(n)
    aug = np.hstack((A.astype(float), I))

    for i in range(n):

        # Pivot element
        pivot = aug[i, i]

        if abs(pivot) < 1e-10:
            raise ValueError("Matrix is singular and cannot be inverted")

        # Normalize pivot row
        aug[i] = aug[i] / pivot

        # Eliminate other rows
        for j in range(n):
            if j != i:
                factor = aug[j, i]
                aug[j] = aug[j] - factor * aug[i]

    # Extract inverse matrix
    inverse = aug[:, n:]
    return inverse


# ---------- Create Random Matrix ----------
def create_matrix(a, b):
    return np.array([[random.randint(1, 9) for _ in range(b)] for _ in range(a)
        ])
```

```python


# ---------- Main Program ----------
if rank == 0:
    x = int(input("Enter number of rows: "))
    y = int(input("Enter number of cols: "))

    if x != y:
        print("Inverse requires a square matrix.")
        comm.Abort()

    A = create_matrix(x, y)

    if x < 10:
        print("\nGenerated Matrix:")
        print(A)

else:
    A = None


start_time = time.perf_counter()

# Broadcast matrix
A = comm.bcast(A, root=0)

# Each process computes inverse
local_inverse = matrix_inverse(A)

# Gather results
all_inverse = comm.gather(local_inverse, root=0)

end_time = time.perf_counter()


if rank == 0:
    print("\nInverse matrices computed by all processes:")
    for i, inv in enumerate(all_inverse):
        print(f"\nProcess {i}:")
        print(inv)

    print("\nFinal Inverse Matrix:")
    print(all_inverse[0])


elapsed_time = end_time - start_time
print(f"Elapsed time: {elapsed_time:.4f} seconds")
```

4.2 Output

```
labmsc04@labmsc04-H610M-H-V2-DDR4: ~/Desktop...
Elapsed time: 1.0172 seconds
Elapsed time: 1.0206 seconds
Elapsed time: 1.0166 seconds
Elapsed time: 1.0206 seconds
Elapsed time: 1.0181 seconds

Inverse matrices computed by all processes:

Process 0:
[[-0.06866953 -0.09871245  0.24892704]
 [ 0.01287554  0.20600858 -0.17167382]
 [ 0.1888412   0.02145923 -0.18454936]]

Process 1:
[[-0.06866953 -0.09871245  0.24892704]
 [ 0.01287554  0.20600858 -0.17167382]
 [ 0.1888412   0.02145923 -0.18454936]]

Process 2:
[[-0.06866953 -0.09871245  0.24892704]
 [ 0.01287554  0.20600858 -0.17167382]
 [ 0.1888412   0.02145923 -0.18454936]]

Process 3:
[[-0.06866953 -0.09871245  0.24892704]
 [ 0.01287554  0.20600858 -0.17167382]
 [ 0.1888412   0.02145923 -0.18454936]]

Process 4:
[[-0.06866953 -0.09871245  0.24892704]
 [ 0.01287554  0.20600858 -0.17167382]
 [ 0.1888412   0.02145923 -0.18454936]]

Process 5:
[[-0.06866953 -0.09871245  0.24892704]
 [ 0.01287554  0.20600858 -0.17167382]
 [ 0.1888412   0.02145923 -0.18454936]]

Final Inverse Matrix:
[[-0.06866953 -0.09871245  0.24892704]
 [ 0.01287554  0.20600858 -0.17167382]
 [ 0.1888412   0.02145923 -0.18454936]]
Elapsed time: 0.0005 seconds
labmsc04@labmsc04-H610M-H-V2-DDR4:~/Desktop/ND23/HPC$
```

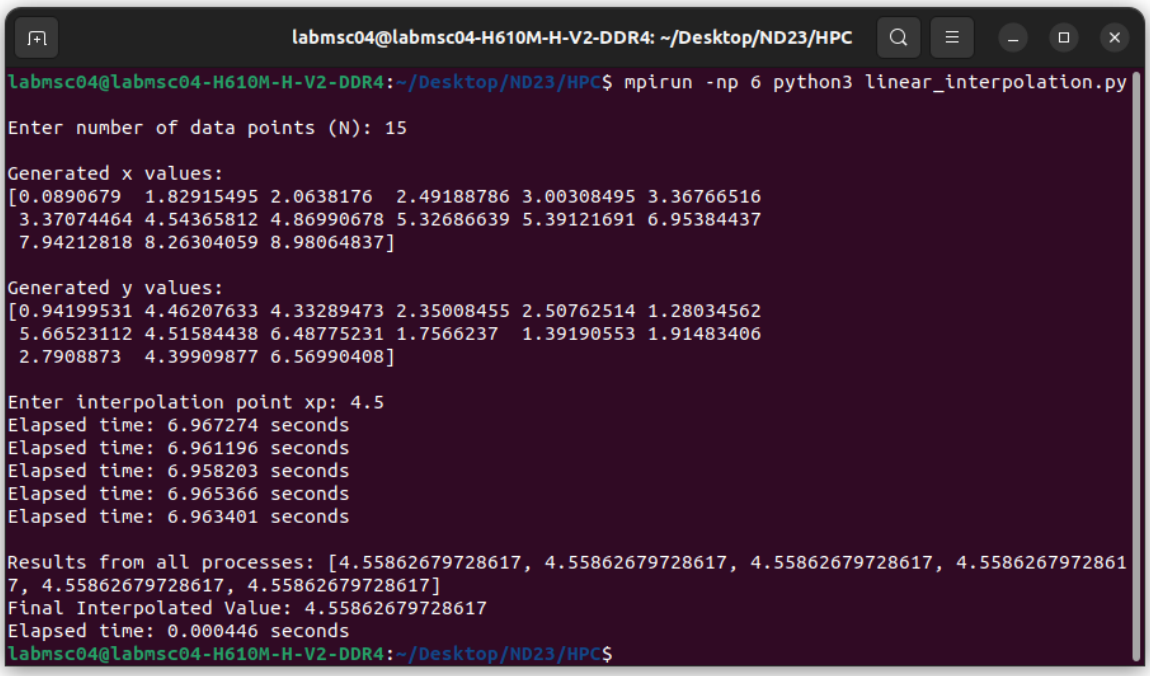
5 Write a program in python to find the linear interpolation of randomly generated data using MPI.

5.1 Python Code

```python
from mpi4py import MPI
import numpy as np
import random
import time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()


# ---------- Linear Interpolation Function ----------
def linear_interpolation(x, y, xp):
    for i in range(len(x) - 1):
        if x[i] <= xp <= x[i+1]:
            yp = y[i] + ((xp - x[i]) * (y[i+1] - y[i])) / (x[i+1] - x[i])
            return yp
    return None


# ---------- Generate Random Data ----------
def generate_data(n):
    x = sorted([random.uniform(0, 9) for _ in range(n)])
    y = [random.uniform(0, 9) for _ in range(n)]
    return np.array(x), np.array(y)


# ---------- Main Program ----------
if rank == 0:
    n = int(input("Enter number of data points (N): "))

    x, y = generate_data(n)

    print("\nGenerated x values:")
    print(x)

    print("\nGenerated y values:")
    print(y)

    xp = float(input("\nEnter interpolation point xp: "))

else:
    x = None
    y = None
    xp = None

```

```python

start_time = time.perf_counter()

# Broadcast data
x = comm.bcast(x, root=0)
y = comm.bcast(y, root=0)
xp = comm.bcast(xp, root=0)

# Each process computes interpolation
local_result = linear_interpolation(x, y, xp)

# Gather results
results = comm.gather(local_result, root=0)

end_time = time.perf_counter()


if rank == 0:
    print("\nResults from all processes:", results)
    print("Final Interpolated Value:", results[0])

elapsed_time = end_time - start_time
print(f"Elapsed time: {elapsed_time:.6f} seconds")
```

5.2 Output



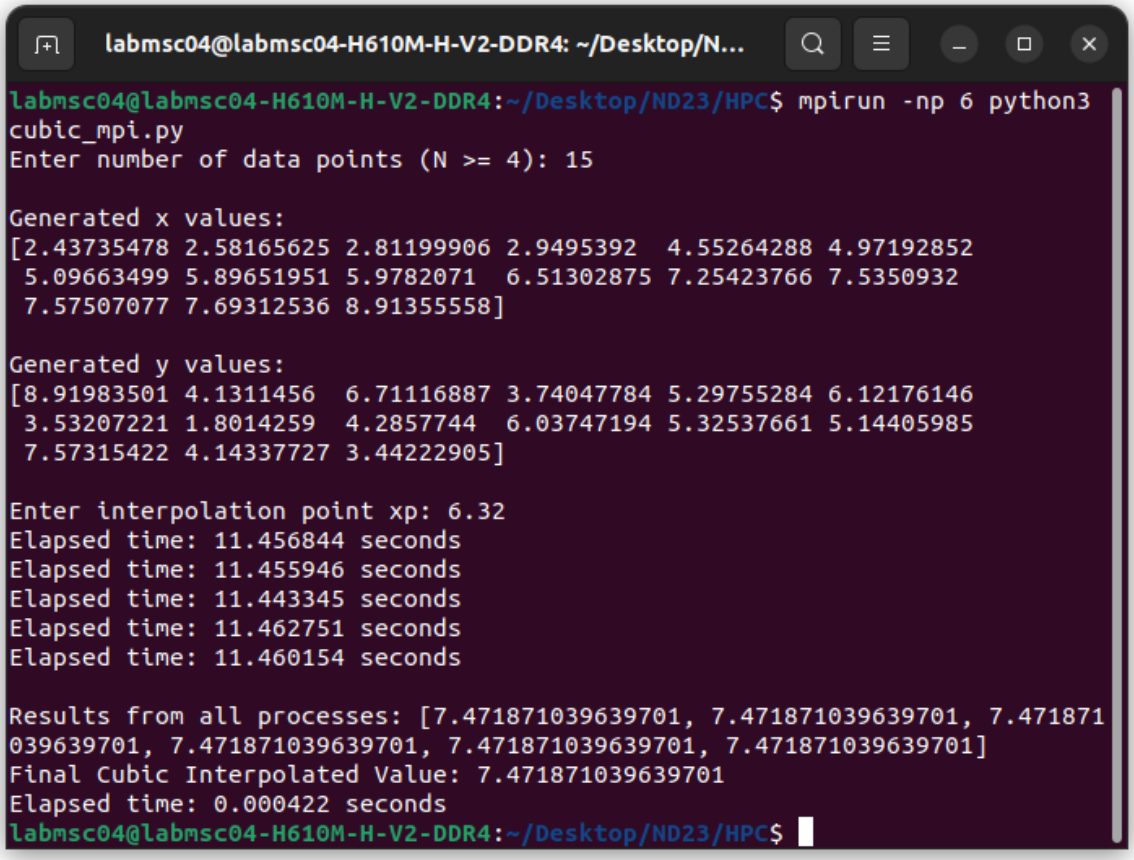
6 Write a program in python to find the cubic interpolation of randomly generated data using MPI.

6.1 Python Code

```python
from mpi4py import MPI
import numpy as np
import random
import time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# ---------- Cubic Interpolation (Lagrange form) ----------
def cubic_interpolation(x, y, xp):
    n = len(x)
    # find segment where xp lies
    for i in range(1, n-2):
        if x[i] <= xp <= x[i+1]:
            # take 4 points
            x0, x1, x2, x3 = x[i-1], x[i], x[i+1], x[i+2]
            y0, y1, y2, y3 = y[i-1], y[i], y[i+1], y[i+2]
            # Lagrange cubic interpolation
            L0 = ((xp-x1)*(xp-x2)*(xp-x3))/((x0-x1)*(x0-x2)*(x0-x3))
            L1 = ((xp-x0)*(xp-x2)*(xp-x3))/((x1-x0)*(x1-x2)*(x1-x3))
            L2 = ((xp-x0)*(xp-x1)*(xp-x3))/((x2-x0)*(x2-x1)*(x2-x3))
            L3 = ((xp-x0)*(xp-x1)*(xp-x2))/((x3-x0)*(x3-x1)*(x3-x2))
            yp = y0*L0 + y1*L1 + y2*L2 + y3*L3
            return yp
    return None

# ---------- Generate Random Data ----------
def generate_data(n):
    x = sorted([random.uniform(0, 9) for _ in range(n)])
    y = [random.uniform(0, 9) for _ in range(n)]
    return np.array(x), np.array(y)

# ---------- Main Program ----------
if rank == 0:
    n = int(input("Enter number of data points (N >= 4): "))
    x, y = generate_data(n)
    print("\nGenerated x values:")
    print(x)
    print("\nGenerated y values:")
    print(y)
    xp = float(input("\nEnter interpolation point xp: "))

else:
    x = None
```

```python
    y = None
    xp = None
start_time = time.perf_counter()
# Broadcast data
x = comm.bcast(x, root=0)
y = comm.bcast(y, root=0)
xp = comm.bcast(xp, root=0)
# Each process computes interpolation
local_result = cubic_interpolation(x, y, xp)
# Gather results
results = comm.gather(local_result, root=0)

end_time = time.perf_counter()

if rank == 0:
    print("\nResults from all processes:", results)
    print("Final Cubic Interpolated Value:", results[0])

elapsed_time = end_time - start_time
print(f"Elapsed time: {elapsed_time:.6f} seconds")
```

6.2 Output



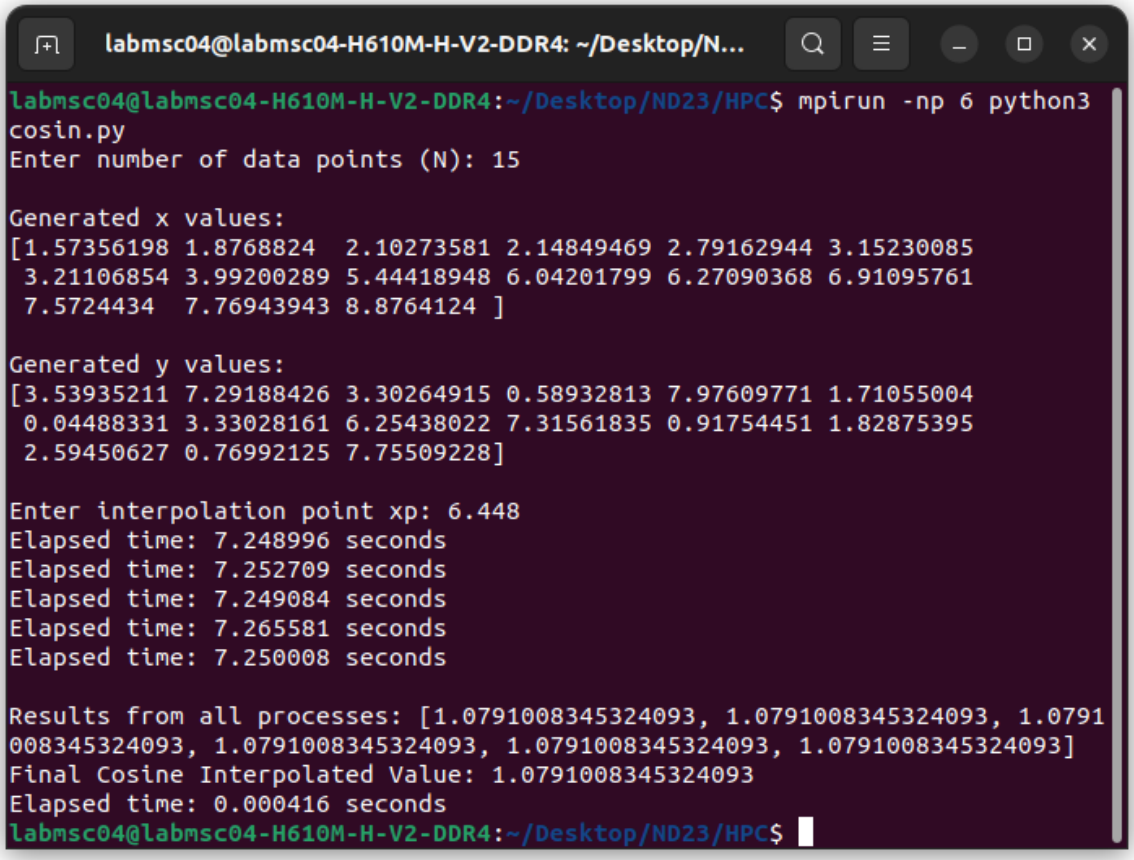
7 Write a program in python to find the cosine interpolation of randomly generated data using MPI.

7.1 Python Code

```python
from mpi4py import MPI
import numpy as np
import random
import time
import math

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# ---------- Cosine Interpolation Function ----------
def cosine_interpolation(x, y, xp):
    for i in range(len(x) - 1):
        if x[i] <= xp <= x[i+1]:

            t = (xp - x[i]) / (x[i+1] - x[i])
            mu = (1 - math.cos(t * math.pi)) / 2

            yp = y[i]*(1-mu) + y[i+1]*mu
            return yp

    return None

# ---------- Generate Random Data ----------
def generate_data(n):
    x = sorted([random.uniform(0, 9) for _ in range(n)])
    y = [random.uniform(0, 9) for _ in range(n)]
    return np.array(x), np.array(y)

# ---------- Main Program ----------
if rank == 0:
    n = int(input("Enter number of data points (N): "))
    x, y = generate_data(n)
    print("\nGenerated x values:")
    print(x)
    print("\nGenerated y values:")
    print(y)
    xp = float(input("\nEnter interpolation point xp: "))
else:
    x = None
    y = None
    xp = None

start_time = time.perf_counter()

```

```python
# Broadcast data
x = comm.bcast(x, root=0)
y = comm.bcast(y, root=0)
xp = comm.bcast(xp, root=0)

# Each process computes interpolation
local_result = cosine_interpolation(x, y, xp)

# Gather results
results = comm.gather(local_result, root=0)

end_time = time.perf_counter()


if rank == 0:
    print("\nResults from all processes:", results)
    print("Final Cosine Interpolated Value:", results[0])

elapsed_time = end_time - start_time
print(f"Elapsed time: {elapsed_time:.6f} seconds")
```

7.2 Output



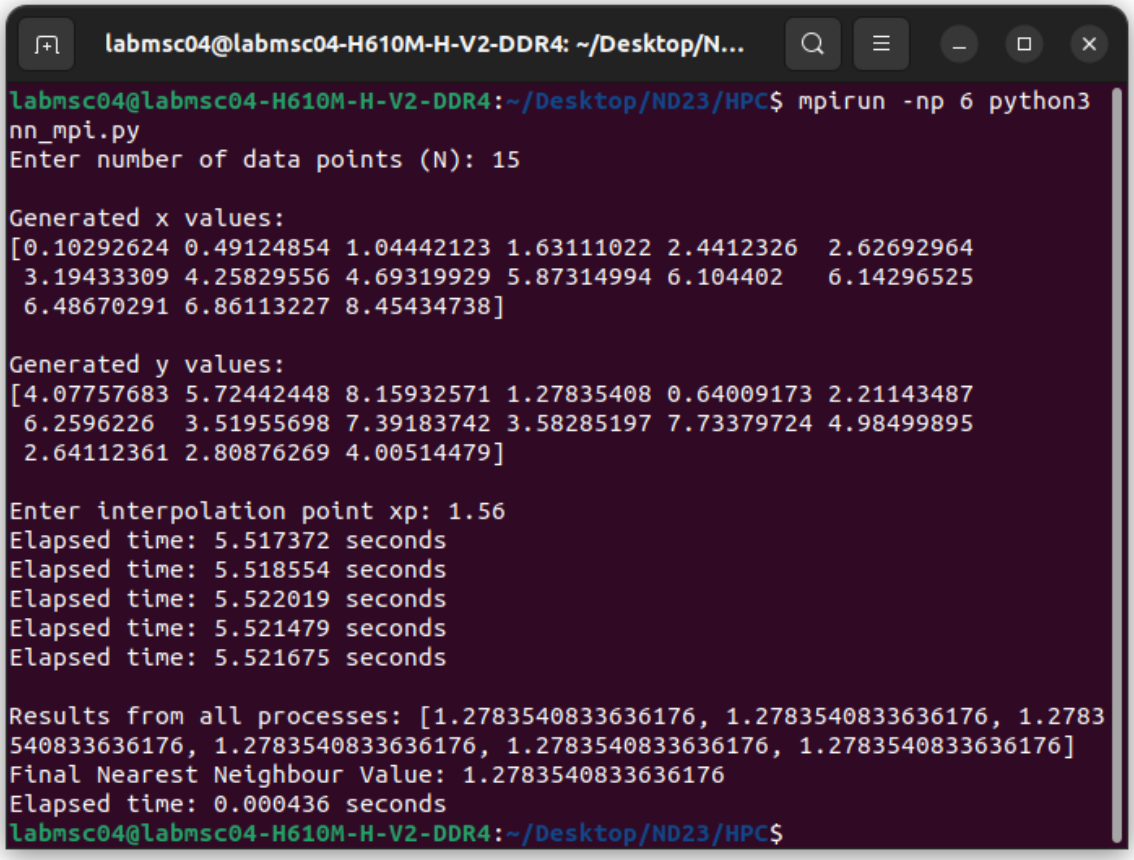
8 Write a program in python to find the Nearest Neighbor interpolation of randomly generated data using MPI.

8.1 Python Code

```python
from mpi4py import MPI
import numpy as np
import random
import time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# ---------- Nearest Neighbour Interpolation ----------
def nearest_neighbor(x, y, xp):
    nearest_index = 0
    min_dist = abs(x[0] - xp)
    for i in range(1, len(x)):
        dist = abs(x[i] - xp)
        if dist < min_dist:
            min_dist = dist
            nearest_index = i
    return y[nearest_index]

# ---------- Generate Random Data ----------
def generate_data(n):
    x = sorted([random.uniform(0, 9) for _ in range(n)])
    y = [random.uniform(0, 9) for _ in range(n)]
    return np.array(x), np.array(y)

# ---------- Main Program ----------
if rank == 0:
    n = int(input("Enter number of data points (N): "))

    x, y = generate_data(n)
    print("\nGenerated x values:")
    print(x)
    print("\nGenerated y values:")
    print(y)
    xp = float(input("\nEnter interpolation point xp: "))

else:
    x = None
    y = None
    xp = None
start_time = time.perf_counter()

# Broadcast data
```

```python
x = comm.bcast(x, root=0)
y = comm.bcast(y, root=0)
xp = comm.bcast(xp, root=0)

# Each process computes interpolation
local_result = nearest_neighbor(x, y, xp)

# Gather results
results = comm.gather(local_result, root=0)

end_time = time.perf_counter()


if rank == 0:
    print("\nResults from all processes:", results)
    print("Final Nearest Neighbour Value:", results[0])

elapsed_time = end_time - start_time
print(f"Elapsed time: {elapsed_time:.6f} seconds")
```

8.2 Output



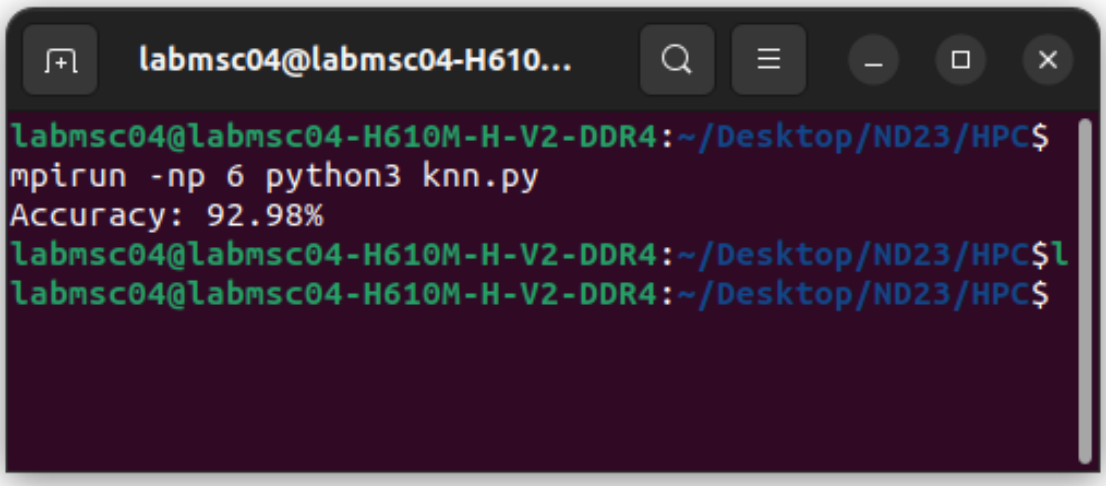
9 Write a program in python to implement KNN Algorithm and find the accuracy using a dataset using MPI.

9.1 Python Code

```python
from mpi4py import MPI
import pandas as pd
import numpy as np

# Initialize MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()
# Load and preprocess dataset
if rank == 0:
    df = pd.read_csv("Cancer_Data.csv")
    # Drop unnecessary columns
    df = df.drop(columns=["id", "Unnamed: 32"], errors='ignore')
    # Convert diagnosis to numeric (M=1, B=0)
    df["diagnosis"] = df["diagnosis"].map({"M": 1, "B": 0})
    # Shuffle dataset
    df = df.sample(frac=1, random_state=42).reset_index(drop=True)
    # Split into features and labels
    X = df.drop("diagnosis", axis=1).values
    y = df["diagnosis"].values
    # Normalize features
    X = (X - X.mean(axis=0)) / X.std(axis=0)
    # Train-test split (80-20)
    split = int(0.8 * len(X))
    X_train, X_test = X[:split], X[split:]
    y_train, y_test = y[:split], y[split:]
else:
    X_train = None
    X_test = None
    y_train = None
    y_test = None

# Broadcast data to all processes
X_train = comm.bcast(X_train, root=0)
y_train = comm.bcast(y_train, root=0)
X_test = comm.bcast(X_test, root=0)
y_test = comm.bcast(y_test, root=0)

# KNN Implementation
def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

def knn_predict(X_train, y_train, x, k=5):
```

```python
    distances = np.array([euclidean_distance(x, x_train) for x_train in X_train
        ])
    k_indices = np.argsort(distances)[:k]
    k_labels = y_train[k_indices]

    # Majority vote
    return np.bincount(k_labels).argmax()
# Parallel Prediction
n_test = len(X_test)
# Split test data across processes
chunk_size = n_test // size
start = rank * chunk_size
end = (rank + 1) * chunk_size if rank != size - 1 else n_test
X_local = X_test[start:end]
y_local = y_test[start:end]
predictions_local = []
for x in X_local:
    pred = knn_predict(X_train, y_train, x, k=5)
    predictions_local.append(pred)
predictions_local = np.array(predictions_local)
# Gather results
predictions = comm.gather(predictions_local, root=0)
# Evaluate
if rank == 0:
    predictions = np.concatenate(predictions)
    accuracy = np.mean(predictions == y_test)
    print(f"Accuracy: {accuracy * 100:.2f}%")
```

9.2 Output



10 Write a program in python to implement K Means Clustering Algorithm and find the accuracy using a dataset using MPI.

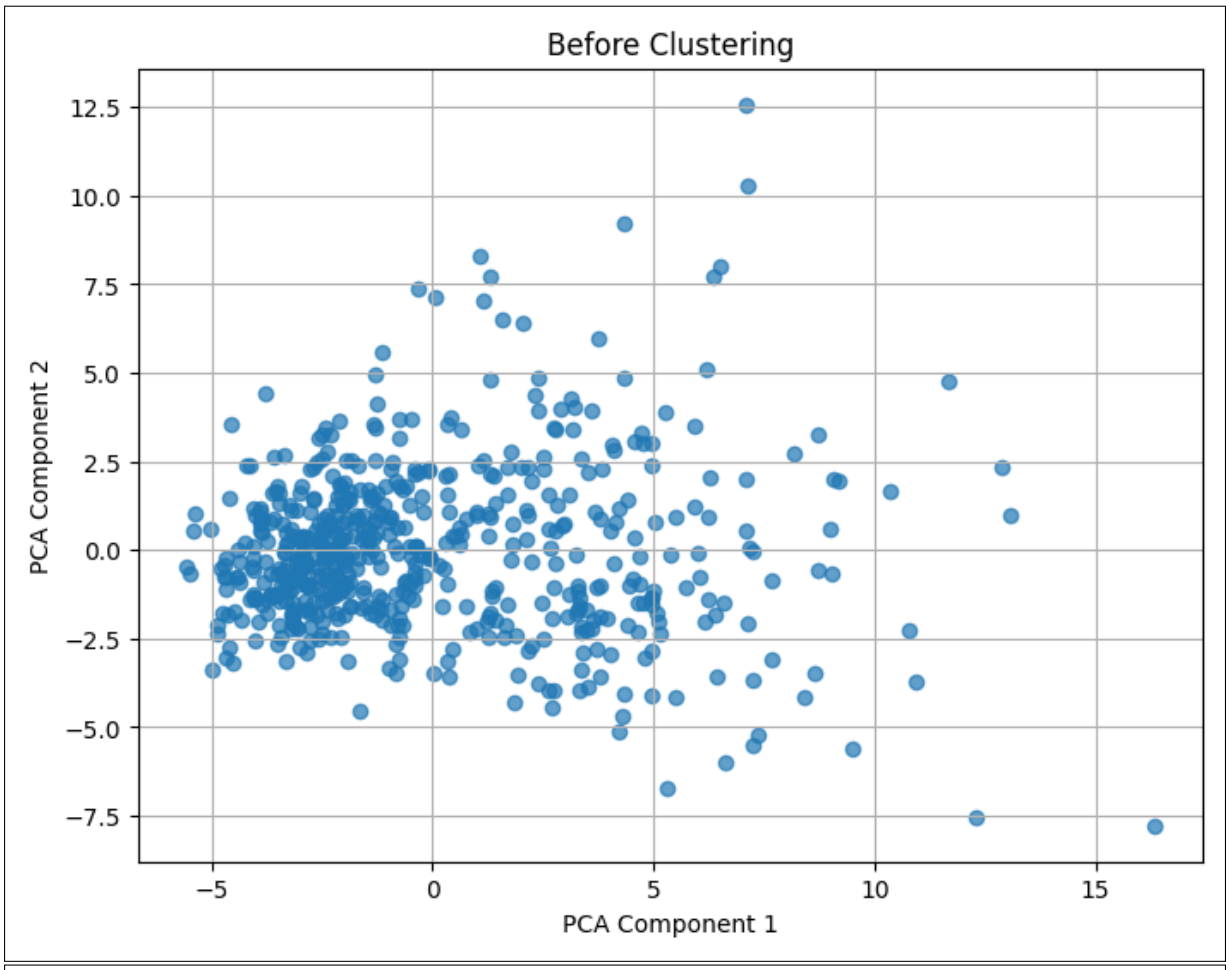
10.1 Python Code

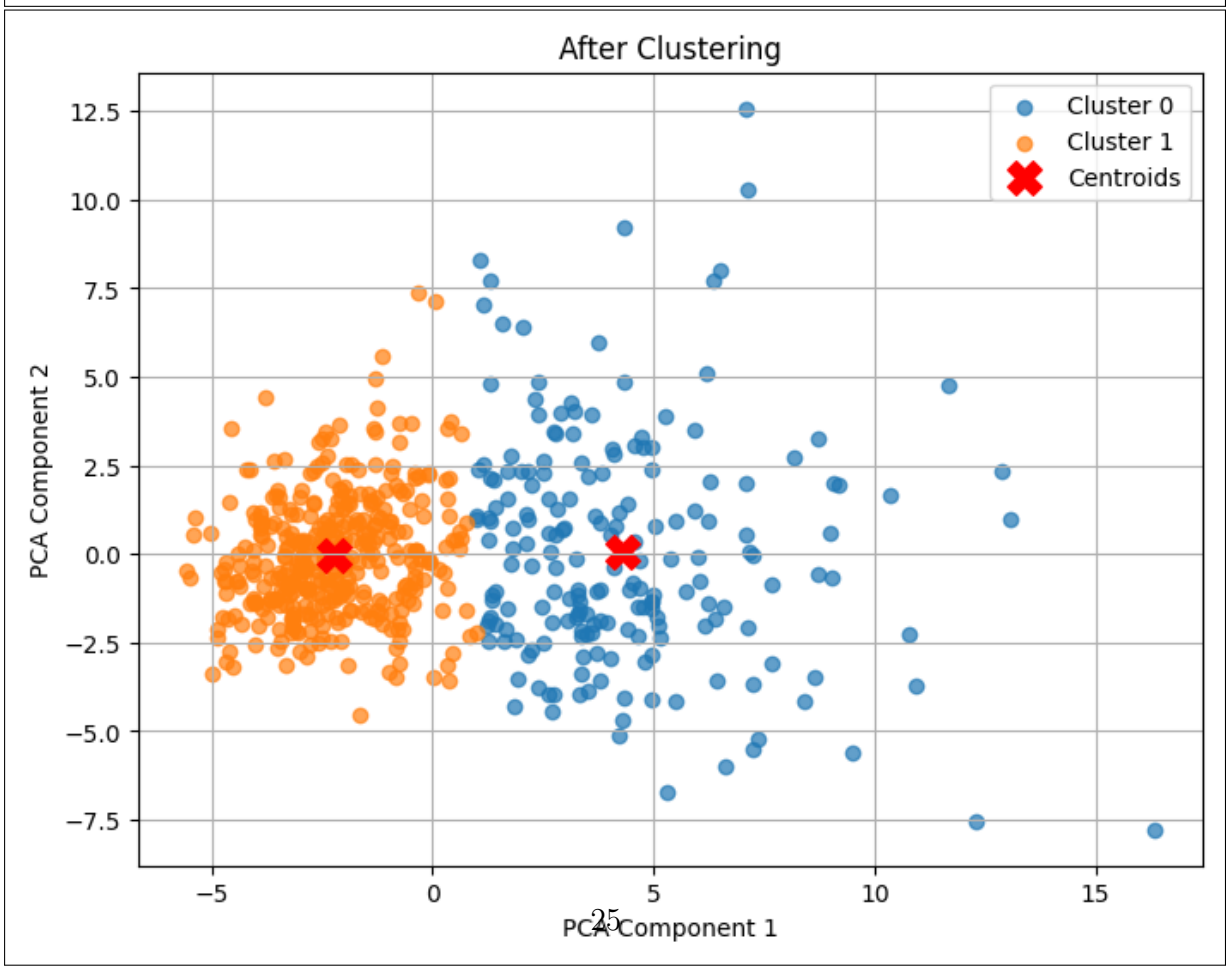
```python
from mpi4py import MPI
import pandas as pd
import numpy as np
import sys
from sklearn.decomposition import PCA
# MPI Setup
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()
# Import matplotlib only on master process
if rank == 0:
    import matplotlib.pyplot as plt
# Parameters
try:
    K = int(sys.argv[1])
except (IndexError, ValueError):
    K = 2 # default number of clusters
max_iters = 100
# Load Data (only master)
if rank == 0:
    df = pd.read_csv("/content/Cancer_Data.csv")
    # Drop unnecessary columns
    df = df.drop(columns=["id", "Unnamed: 32"], errors="ignore")
    # Convert diagnosis column to numeric
    df["diagnosis"] = df["diagnosis"].map({"M": 1, "B": 0})
    # Features only
    X = df.drop("diagnosis", axis=1).values
    # Normalize the dataset
    X = (X - X.mean(axis=0)) / X.std(axis=0)
    n_samples = X.shape[0]
    # PCA for 2D visualization
    pca = PCA(n_components=2)
    X_2d = pca.fit_transform(X)
    # Randomly initialize centroids
    indices = np.random.choice(n_samples, K, replace=False)
    centroids = X[indices]
else:
    X = None
    X_2d = None
    centroids = None
    pca = None
# Broadcast Data to All Processes
X = comm.bcast(X, root=0)
centroids = comm.bcast(centroids, root=0)
```

```python
n_samples = X.shape[0]
# Divide Data Among Processes
chunk = n_samples // size
start = rank * chunk
end = (rank + 1) * chunk if rank != size - 1 else n_samples
X_local = X[start:end]
# K-Means Clustering
for iteration in range(max_iters):
    # Step 1: Assign local points to nearest centroid
    local_labels = []
    for x in X_local:
        distances = np.sqrt(np.sum((centroids - x) ** 2, axis=1))
        cluster = np.argmin(distances)
        local_labels.append(cluster)
    local_labels = np.array(local_labels)
    # Step 2: Compute local sum and count for each cluster
    local_sum = np.zeros_like(centroids)
    local_count = np.zeros(K)
    for i, label in enumerate(local_labels):
        local_sum[label] += X_local[i]
        local_count[label] += 1
    # Step 3: Reduce local sums and counts into global sums and counts
    global_sum = np.zeros_like(centroids)
    global_count = np.zeros(K)
    comm.Allreduce(local_sum, global_sum, op=MPI.SUM)
    comm.Allreduce(local_count, global_count, op=MPI.SUM)
    # Step 4: Update centroids
    new_centroids = np.copy(centroids)
    for k in range(K):
        if global_count[k] > 0:
            new_centroids[k] = global_sum[k] / global_count[k]
    # Step 5: Check convergence
    if np.allclose(centroids, new_centroids, atol=1e-4):
        centroids = new_centroids
        break
    centroids = new_centroids
# Final Cluster Labels
local_labels = []
for x in X_local:
    distances = np.sqrt(np.sum((centroids - x) ** 2, axis=1))
    cluster = np.argmin(distances)
    local_labels.append(cluster)
local_labels = np.array(local_labels)
# Gather final labels to master
all_labels = comm.gather(local_labels, root=0)
# Output and Plots
if rank == 0:
    final_labels = np.concatenate(all_labels)
    print("\nFinal Centroids:\n", centroids)
    # Convert final centroids into PCA 2D space
    centroid_2d = pca.transform(centroids)
```

```python
    # Plot 1: Before Clustering
    plt.figure(figsize=(8, 6))
    plt.scatter(X_2d[:, 0], X_2d[:, 1], alpha=0.7)
    plt.title("Before Clustering")
    plt.xlabel("PCA Component 1")
    plt.ylabel("PCA Component 2")
    plt.grid(True)
    plt.show()
    # To save instead of show, use:
    # plt.savefig("before_clustering.png")
    # plt.close()
    # Plot 2: After Clustering
    plt.figure(figsize=(8, 6))
    for k in range(K):
        plt.scatter(
            X_2d[final_labels == k, 0],
            X_2d[final_labels == k, 1],
            alpha=0.7,
            label=f"Cluster {k}"
        )
    plt.scatter(
        centroid_2d[:, 0],
        centroid_2d[:, 1],
        marker='X',
        s=200,
        c='red',
        label='Centroids'
    )
    plt.title("After Clustering")
    plt.xlabel("PCA Component 1")
    plt.ylabel("PCA Component 2")
    plt.legend()
    plt.grid(True)
    plt.show()
    # To save instead of show, use:
    # plt.savefig("after_clustering.png")
    # plt.close()
```

10.2 Output


Before Clustering
PCA Component 2
PCA Component 1
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−5
0
5
10
15


After Clustering
Cluster 0
Cluster 1
Centroids
PCA Component 2
PCA Component 1
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−5
0
5
10
15

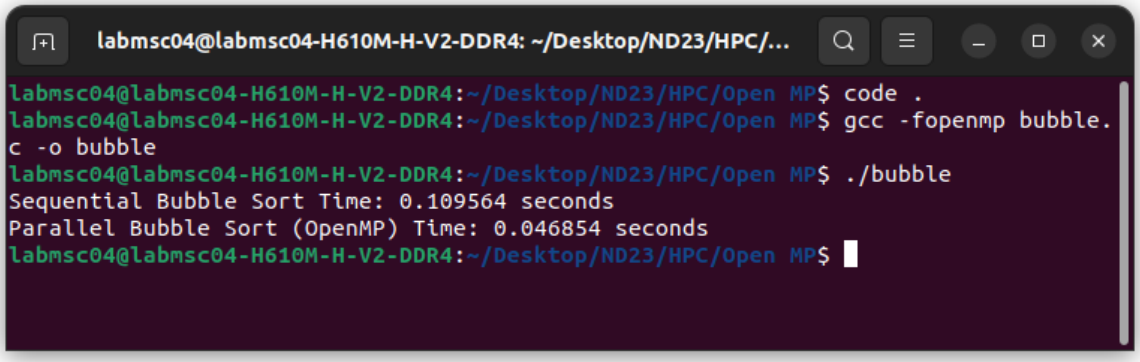
11 Write a program to implement Bubble sort using OpenMP in C Programming

11.1 C Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
// Function to swap two elements
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
// Sequential Bubble Sort
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(&arr[j], &arr[j + 1]);
            }
        }
    }
}
// Parallel Bubble Sort using OpenMP (Odd-Even Sort)
void parallelBubbleSort(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        // Even phase
        #pragma omp parallel for
        for (int j = 0; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1]) {
                swap(&arr[j], &arr[j + 1]);
            }
        }
        // Odd phase
        #pragma omp parallel for
        for (int j = 1; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1]) {
                swap(&arr[j], &arr[j + 1]);
            }
        }
    }
}
// Function to copy array
void copyArray(int source[], int dest[], int n) {
    for (int i = 0; i < n; i++) {
        dest[i] = source[i];
    }
}
int main() {
```

```c
    int n = 10000; // size of array
    int *arr1 = (int *)malloc(n * sizeof(int));
    int *arr2 = (int *)malloc(n * sizeof(int));
    // Generate random data
    for (int i = 0; i < n; i++) {
        arr1[i] = rand() % 10000;
    }
    copyArray(arr1, arr2, n);
    double start, end;
    // Sequential timing
    start = omp_get_wtime();
    bubbleSort(arr1, n);
    end = omp_get_wtime();
    printf("Sequential Bubble Sort Time: %f seconds\n", end - start);
    // Parallel timing
    start = omp_get_wtime();
    parallelBubbleSort(arr2, n);
    end = omp_get_wtime();
    printf("Parallel Bubble Sort (OpenMP) Time: %f seconds\n", end - start);
    free(arr1);
    free(arr2);
    return 0;
}
```

11.2 Output



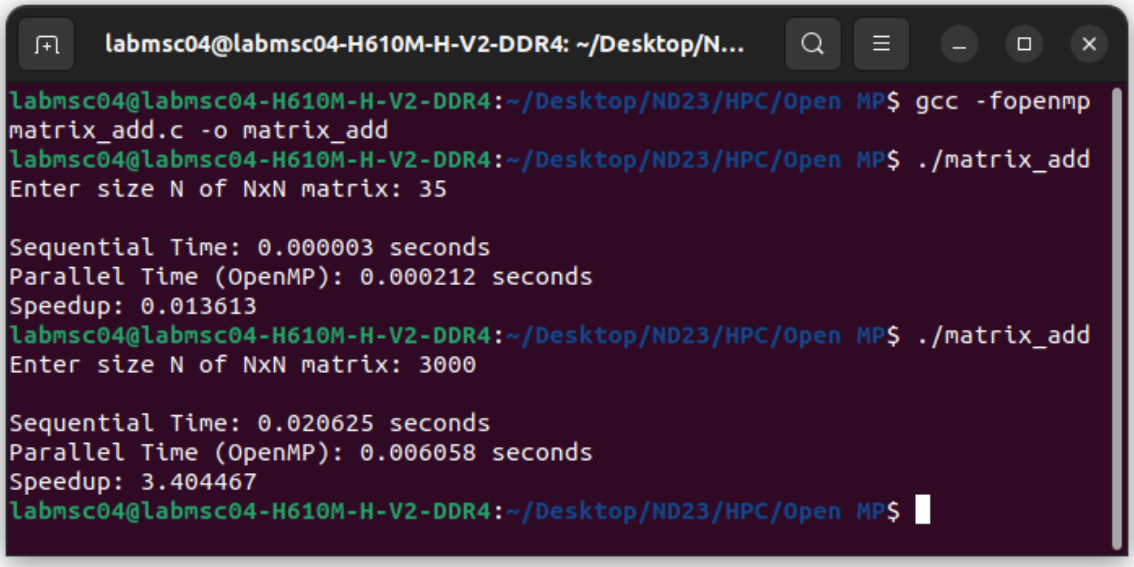
12 Write a program to implement two matrix addition using OpenMP in C Programming

12.1 C Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>
int main() {
    int n;
    printf("Enter size N of NxN matrix: ");
    scanf("%d", &n);
    // Dynamic allocation
    int **A = (int **)malloc(n * sizeof(int *));
    int **B = (int **)malloc(n * sizeof(int *));
    int **C_seq = (int **)malloc(n * sizeof(int *));
    int **C_par = (int **)malloc(n * sizeof(int *));
    for (int i = 0; i < n; i++) {
        A[i] = (int *)malloc(n * sizeof(int));
        B[i] = (int *)malloc(n * sizeof(int));
        C_seq[i] = (int *)malloc(n * sizeof(int));
        C_par[i] = (int *)malloc(n * sizeof(int));
    }
    // Seed for random numbers
    srand(time(NULL));
    // Generate random matrices (values 099)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            A[i][j] = rand() % 100;
            B[i][j] = rand() % 100;
        } }
    double start, end;
    // Sequential addition
    start = omp_get_wtime();
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C_seq[i][j] = A[i][j] + B[i][j];
    end = omp_get_wtime();
    double seq_time = end - start;
    start = omp_get_wtime();
    #pragma omp parallel for collapse(2)
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C_par[i][j] = A[i][j] + B[i][j];
    end = omp_get_wtime();
    double par_time = end - start;
    if (n <= 5) {
        printf("\nMatrix A:\n");
        for (int i = 0; i < n; i++) {
```

```c
            for (int j = 0; j < n; j++)
                printf("%d ", A[i][j]);
            printf("\n");
        }
        printf("\nMatrix B:\n");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++)
                printf("%d ", B[i][j]);
            printf("\n");
        }
        printf("\nResult Matrix:\n");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++)
                printf("%d ", C_seq[i][j]);
            printf("\n");
        } }
    // Time comparison
    printf("\nSequential Time: %f seconds\n", seq_time);
    printf("Parallel Time (OpenMP): %f seconds\n", par_time);
    // Speedup
    printf("Speedup: %f\n", seq_time / par_time);
    // Free memory
    for (int i = 0; i < n; i++) {
        free(A[i]); free(B[i]);
        free(C_seq[i]); free(C_par[i]);
    }
    free(A); free(B); free(C_seq); free(C_par);
    return 0;
}
```

12.2 Output

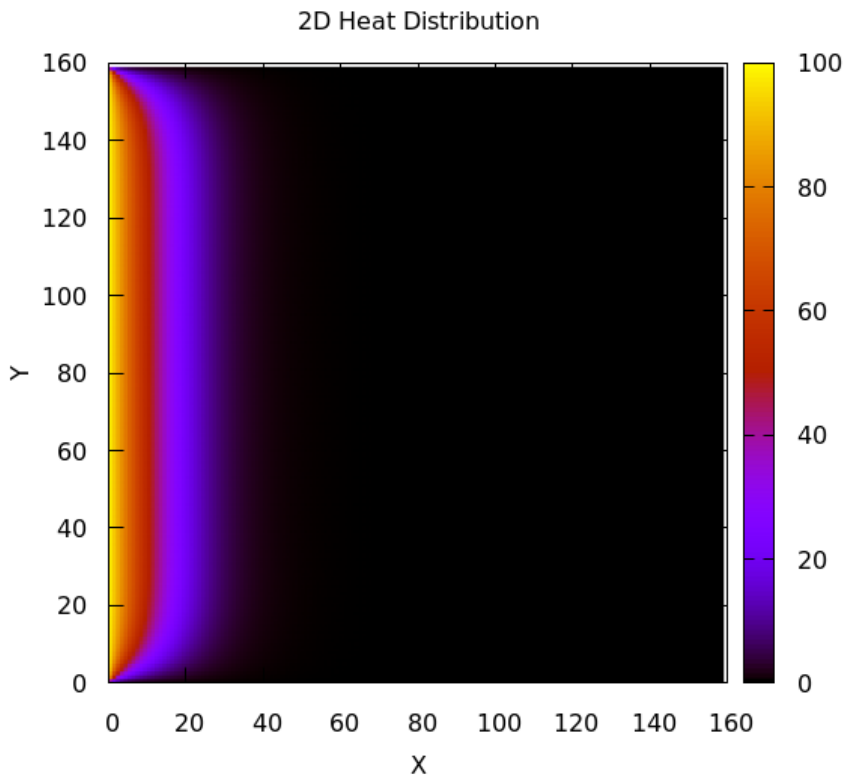


13 Write a program to implement 2d heat equation using OpenMP and MPI in C Programming

13.1 C Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
#include <omp.h>
#define N 160
#define MAX_ITER 500

int main(int argc, char *argv[]) {
    int rank, size;
    int i, j, iter;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (N % size != 0) {
        if (rank == 0)
            printf("Error: N must be divisible by number of processes\n");
        MPI_Finalize();
        return 1;
    }
    int rows = N / size;
    // Allocate contiguous memory
    double *u = (double *)malloc((rows + 2) * N * sizeof(double));
    double *u_new = (double *)malloc((rows + 2) * N * sizeof(double));
    // Access macro
    #define U(i,j) u[(i)*N + (j)]
    #define U_NEW(i,j) u_new[(i)*N + (j)]
    // Initialize
    for (i = 0; i < rows + 2; i++)
        for (j = 0; j < N; j++)
            U(i,j) = 0.0;
    // Top boundary (global)
    if (rank == 0) {
        for (j = 0; j < N; j++)
            U(1,j) = 100.0;
    }
    MPI_Status status;
    for (iter = 0; iter < MAX_ITER; iter++) {
        // Exchange ghost rows
        if (rank > 0) {
            MPI_Sendrecv(&U(1,0), N, MPI_DOUBLE, rank-1, 0,
                         &U(0,0), N, MPI_DOUBLE, rank-1, 0,
                         MPI_COMM_WORLD, &status);
        }
        if (rank < size-1) {
            MPI_Sendrecv(&U(rows,0), N, MPI_DOUBLE, rank+1, 0,
```

```
                        &U(rows+1,0), N, MPI_DOUBLE, rank+1, 0,
                        MPI_COMM_WORLD, &status);
        }
        // Compute
        #pragma omp parallel for private(j)
        for (i = 1; i <= rows; i++) {
            for (j = 1; j < N-1; j++) {
                U_NEW(i,j) = 0.25 * (U(i+1,j) + U(i-1,j) +
                                    U(i,j+1) + U(i,j-1));
            }
        }
        // Preserve boundaries
        if (rank == 0) {
            for (j = 0; j < N; j++)
                U_NEW(1,j) = 100.0;
        }
        for (i = 1; i <= rows; i++) {
            U_NEW(i,0) = 0.0;
            U_NEW(i,N-1) = 0.0;
        }
        // Copy back
        #pragma omp parallel for private(j)
        for (i = 1; i <= rows; i++) {
            for (j = 0; j < N; j++) {
                U(i,j) = U_NEW(i,j);
            }
        }
    }
    // Gather results
    double *global = NULL;
    if (rank == 0)
        global = (double *)malloc(N * N * sizeof(double));

    MPI_Gather(&U(1,0), rows * N, MPI_DOUBLE,
               global, rows * N, MPI_DOUBLE,
               0, MPI_COMM_WORLD);
    // Output
    if (rank == 0) {
        FILE *fp = fopen("heat.dat", "w");

        for (i = 0; i < N; i++) {
            for (j = 0; j < N; j++) {
                fprintf(fp, "%d %d %f\n", i, j, global[i*N + j]);
            }
            fprintf(fp, "\n");
        }
        fclose(fp);
        printf("Data written to heat.dat\n");
    }
    free(u);
    free(u_new);
```

```
    if (rank == 0) free(global);
    MPI_Finalize();
    return 0;
}
```

13.2 Output